Projeto QA — Academia Fitness

Para membros da equipe que precisam apresentar o projeto

Lucas Mendes · Matheus Goes · Wendell Barboza

Qualidade de Software — Prof. Icaro Ferreira · SENAC

1. O que é este projeto?

Desenvolvemos um sistema de gerenciamento de uma **academia de ginástica** e criamos uma automação que testa se o sistema funciona corretamente. Essa automação abre o navegador, clica nos botões, preenche formulários e verifica se o sistema se comporta como esperado — tudo sozinho, sem intervenção humana.

O foco não é o sistema em si (a academia), mas sim demonstrar o **processo de QA**: como testamos um software, classificamos os defeitos e decidimos se ele pode ser liberado.

Tecnologias usadas:

Tecnologia	Para que serve
Python	Linguagem de programação principal
Flask	Servidor web que roda o sistema da academia
Playwright	Robô que abre o browser e faz os testes automaticamente
HTML/CSS	Interface visual do sistema (o site que aparece na tela)

2. As 4 funcionalidades do sistema

O sistema da academia tem 4 funções principais. Para cada uma delas existem **regras de negócio** — ou seja, regras que o sistema deve obedecer:

RF01 — Cadastro de Membros

Registrar um novo aluno na academia informando nome, CPF, idade e plano.

- CPF único — não pode cadastrar dois alunos com o mesmo CPF
- Idade mínima de 16 anos
- Plano deve ser: mensal (30 dias), trimestral (90 dias) ou anual (365 dias)

RF02 — Check-in

Registrar a entrada de um aluno na academia pelo CPF.

- Só membros cadastrados podem fazer check-in
- Matrícula cancelada não pode fazer check-in
- Matrícula vencida não pode fazer check-in

RF03 — Cancelamento de Matrícula

Cancelar a matrícula de um aluno, impedindo futuros check-ins.

- O CPF informado precisa existir no sistema

RF04 — Renovação de Plano

Reativar a matrícula de um aluno cancelado e definir um novo plano.

- O CPF deve existir no sistema
- O plano deve ser válido
- A renovação reativa automaticamente uma matrícula cancelada

3. Os 12 casos de teste (em linguagem simples)

Um **caso de teste** é uma situação que verificamos para saber se o sistema se comporta corretamente. Cada teste tem um resultado esperado e um resultado obtido. Se baterem: **PASS**. Se não baterem: **FAIL**.

ID	O que testa	Esperado	Criticidade
TC001	A página abre corretamente com título e 5 abas	Página carrega OK	BAIXA
TC002	Cadastrar um aluno com dados válidos	Cadastro aceito	ALTA
TC003	Tentar cadastrar com CPF que já existe	Sistema rejeita	ALTA
TC004	Tentar cadastrar aluno com 14 anos (menor de 16)	Sistema rejeita	ALTA
TC005	Tentar enviar plano inválido manipulando o HTML	Servidor rejeita	MÉDIA
TC006	Check-in de aluno com matrícula ativa	Check-in aceito	CRÍTICA
TC007	Cancelar matrícula de um aluno	Cancelado com sucesso	ALTA
TC008	Tentar check-in com matrícula cancelada (BUG)	Sistema rejeita	CRÍTICA
TC009	Tentar check-in de CPF inexistente	Sistema rejeita	ALTA
TC010	Tentar cancelar CPF inexistente	Sistema rejeita	MÉDIA
TC011	Renovar plano de membro cancelado	Renovado com sucesso	ALTA
TC012	Tentar renovar CPF inexistente	Sistema rejeita	MÉDIA

■ TC008 é o teste mais importante: ele verifica se o sistema impede corretamente o check-in de quem está com matrícula cancelada. É onde o BUG proposital aparece.

4. Conceitos importantes para a apresentação

PASS / FAIL

PASS = o teste passou (o sistema se comportou como esperado). FAIL = o teste falhou (o sistema fez algo errado).

Severidade

Indica o impacto do defeito. CRÍTICA = o sistema não pode ser liberado. ALTA = funcionalidade principal quebrada. MÉDIA = inconveniência. BAIXA = impacto mínimo.

Taxa de aprovação

Percentual de testes que passaram. Ex: 11 de 12 testes passando = 91,7% de aprovação.

Decisão final de QA

LIBERADO = todos passaram. LIBERADO COM RESSALVAS = falhas não críticas. NÃO LIBERAR = há pelo menos uma falha crítica.

Bug proposital

Inserimos um defeito intencional no código para mostrar que a automação consegue identificar problemas. Isso demonstra que os testes funcionam de verdade.

Relatório HTML

Ao final da execução, o programa gera automaticamente um arquivo HTML com todos os resultados, métricas e a decisão de QA.

5. Como rodar o projeto no VS Code

Passo a passo no dia da apresentação:

1	**Abra a pasta ATV-QA no VS Code** File → Open Folder → selecione a pasta "ATV-QA"
2	**Abra o terminal integrado** Menu: Terminal → New Terminal (ou Ctrl + `)
3	**Instale as dependências (apenas na primeira vez)** Digite: pip install flask playwright Depois: playwright install msedge
4	**Execute a automação normal (sem bug)** Digite: python run_qa.py O browser Edge vai abrir automaticamente e realizar todos os testes.
5	**Execute com o bug proposital** Digite: python run_qa.py --bug Observe que o TC008 vai FALHAR e a decisão muda para NÃO LIBERAR.
6	**O relatório abre sozinho** Ao finalizar, o relatório HTML é aberto automaticamente no navegador.

> Não é necessário rodar o app.py separadamente. O comando python run_qa.py já sobe o servidor Flask automaticamente, roda os testes e derruba o servidor no final.

6. Roteiro de apresentação (o que falar)

Se você for sorteado, siga este roteiro. Não precisa decorar — entenda a ideia geral.

1 Apresentar o sistema

"Desenvolvemos um sistema de gerenciamento de uma academia de ginástica. Ele permite cadastrar membros, fazer check-in, cancelar matrículas e renovar planos. O objetivo do projeto não é o sistema em si, mas demonstrar o processo de QA — Garantia da Qualidade de Software."

2 Mostrar as funcionalidades

"O sistema tem 4 funcionalidades principais, cada uma com regras de negócio. Por exemplo, no cadastro: o CPF precisa ser único e o membro precisa ter pelo menos 16 anos. No check-in: só membros com matrícula ativa podem entrar."

3 Executar a automação (sem bug)

"Vou executar agora a automação. O Playwright vai abrir o Edge automaticamente, navegar pelo sistema, preencher os formulários e verificar se os resultados estão corretos." [Execute: python run_qa.py] "Cada popup na tela mostra se o teste passou ou falhou. Verde é PASS, vermelho é FAIL."

4 Mostrar o relatório

"Ao finalizar, o relatório HTML é gerado automaticamente. Aqui vemos as métricas: total de testes, aprovados, reprovados, taxa de aprovação e a decisão final de QA. Como todos os testes passaram, o sistema está LIBERADO."

5 Demonstrar o bug proposital

"Agora vou introduzir um bug intencional para mostrar que a automação detecta defeitos." [Execute: python run_qa.py --bug] "O TC008 falhou — ele deveria bloquear o check-in de uma matrícula cancelada, mas o bug faz o sistema aceitar. Isso é uma falha crítica, então a decisão muda para NÃO LIBERAR."

6 Encerrar

"O ponto central do trabalho é mostrar que compreendemos o processo completo: Requisito → Teste → Execução → Resultado → Identificação de Defeito → Decisão de QA → Relatório."

7. Perguntas que o professor pode fazer

■ Por que vocês escolheram uma academia?

É um sistema simples com regras de negócio claras e fáceis de entender, ideal para demonstrar o processo de QA sem complexidade desnecessária.

■ O que é Playwright?

É uma ferramenta da Microsoft que permite automatizar ações no navegador. Com ela, criamos um 'robô' que abre o site, preenche formulários e clica em botões exatamente como um usuário faria — mas de forma automática e repetível.

■ Por que a severidade do TC008 é CRÍTICA?

Porque permite que alguém com matrícula cancelada acesse a academia. É um problema de segurança e integridade do negócio — não pode ser ignorado.

■ O que acontece se o sistema tiver só falhas MÉDIA?

A decisão seria LIBERADO COM RESSALVAS — o sistema pode ir para produção, mas com ressalvas: os defeitos precisam ser corrigidos em breve.

■ Como o bug é ativado?

Com o flag --bug na linha de comando. Ele edita automaticamente o arquivo sistema_academia.py para desativar a verificação de matrícula cancelada no check-in. Após a execução, o bug é desativado automaticamente para não afetar outros testes.

■ O que é taxa de aprovação?

É o percentual de testes que passaram. Se 11 de 12 testes passaram: 11 ÷ 12 × 100 = 91,7% de aprovação.